

**DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE**

**Total Marks:** \_\_\_\_\_

**Obtained Marks:** \_\_\_\_\_

# **Object-Oriented Programming Techniques**

## **Project Report**

### **AsaanPay Digital Wallet System**

| <b>Names</b>               | <b>Registration Numbers</b> |
|----------------------------|-----------------------------|
| <b>Badar Ahmad Dhindsa</b> | <b>25108238</b>             |
| <b>Talha Maqsood</b>       | <b>25108385</b>             |
| <b>Fahad Malik</b>         | <b>25108240</b>             |

**Submitted To: Sir Hassan Mujtaba**

---

## **1. Introduction:**

Digital wallets have become a very important part of modern life, especially in countries like Pakistan where services such as EasyPaisa and JazzCash are used by millions of people every day. These applications let users store money, send it to others, pay bills, and check their balance without using physical cash or visiting a bank. At their core, all of these systems are built around the same basic ideas: keeping track of user accounts, storing balances, recording transactions, and making sure that only authorized users can access each account. Understanding how to build such a system at a basic level is a very valuable exercise for anyone learning Object-Oriented Programming.

AsaanPay is a simplified but complete simulation of a digital wallet system built entirely in C++ as a console application. It does not connect to any real bank or payment service but it works in a realistic and meaningful way inside the terminal window. A user can register a new wallet account by providing their name, email, and a four-digit PIN. After logging in with the correct name and PIN, the user gets access to a personal menu from which they can top up their wallet balance, cash out money, send money to another registered user with an optional note, and view all past transactions in a numbered list. The user can also change their PIN at any time. The main menu includes a System Info option that shows live statistics for the entire system including the total number of users, total transactions completed, and the combined balance across all accounts.

The project was built to practice and demonstrate Object-Oriented Programming concepts in a real and working context rather than in a theoretical or isolated example. The code uses Encapsulation to protect sensitive user data, Abstraction to hide the internal logic of operations, Arrays of Objects to store all user accounts in one place, Static Variables to track system-wide information, Function Overloading to handle similar tasks with different inputs, Constructors to properly set up User objects, and Getters and Setters to control how data is read and changed from outside the class. All seven of these concepts are active in the code and this report explains each one in full detail.

## **2. Objectives:**

The main objectives of the AsaanPay Digital Wallet System are as follows:

- Allow users to register a new wallet account by entering their full name, email address, and a four-digit PIN, and automatically assign each account a unique ID in the format AP1001, AP1002, and so on.
- Allow users to log in to their account by entering their full name and PIN, and display the personal user menu only after both fields are matched against an existing account in the user array.
- Provide a Top-Up feature so users can add money to their wallet and a Cash-Out feature so users can withdraw money, with both operations validated before they are processed and then recorded in the transaction history.

- Allow users to send money to any other registered user by searching for them using a name, account ID, or email, entering a transfer amount up to a maximum of 50000, and optionally adding a short note that appears in the history of both the sender and the receiver.
- Maintain a numbered transaction history for every user account that records every deposit, withdrawal, and money transfer, and allow the user to display it at any time through the Transaction History option.
- Use an array of 100 User objects as the central data structure of the program and manage all accounts, login checks, and money transfers through this array.
- Use static member variables inside the User class to keep track of the total registered users, total completed transactions, and total combined balance across all wallets, and display this data through the System Info option.
- Use function overloading for the addHistory and sendMoney functions so that both the version with a note and the version without a note can be called using the same function name.
- Use constructors to properly initialize User objects, getters to safely read private data from outside the class, and setters to update the PIN in a controlled way.
- Apply input validation throughout the program to reject non-numeric menu choices, invalid amounts, PINs that are not exactly four digits, emails that contain uppercase letters, and blank name fields.

### **3. System Overview:**

When the program starts, five pre-registered user accounts are created using the parameterized constructor of the User class and stored in a fixed-size array of 100 User objects. Each account has a name, email, PIN, an auto-generated account ID, and a starting balance. After this setup is complete, the main program loop begins and the main menu appears. The main menu offers four options which are Login, Create Wallet, System Info, and Exit. This loop keeps running until the user selects Exit.

When Login is selected, the user enters their full name and PIN. The program loops through the user array and checks if any account matches both fields. If a match is found, the index of that user is stored and the user menu is displayed. The user menu shows the account name, ID, and current balance at the top and then lists the available actions which are Top-Up, Cash-Out, Send Money, Transaction History, Change PIN, and Logout. The session stays active on this menu until the user logs out, which resets the current index and brings back the main menu.

The Create Wallet option runs a step-by-step registration process where the name, email, and PIN are each validated before the next field is asked for. Once all fields are accepted, a new User object is created using the parameterized constructor, added to the array, and the static system counters are updated. The Send Money feature works by taking a search term, finding matching users in the array using the searchUsers function, letting the sender pick a recipient, asking for an amount and an optional note, showing a confirmation screen, and completing the transfer after the user confirms. The System Info option reads the static variables inside the User class using the static getter functions and displays the total users, total transactions, and total system balance.

## **4. Object-Oriented Concepts Used:**

### **4.1 Encapsulation:**

Encapsulation is the concept of keeping the internal data of a class hidden from the outside world so that it cannot be directly accessed or changed by code outside the class. In this project, all the important and sensitive data of a user account is declared as private inside the User class. This private data includes the name, the PIN, the email address, the account ID, the balance, the transaction history array, and the count of how many history entries have been stored. None of these variables can be touched directly from outside the class. The code in main() or in any other function outside the User class cannot read or write to these fields without going through the public member functions that the class provides. This is enforced by the private keyword in the class definition. The benefit of this approach is that the data is always protected. For example, the balance cannot just be set to any arbitrary number from somewhere in main(). The only ways to change the balance are through deposit(), withdraw(), or sendMoney(), and all three of these functions check whether the requested operation is valid before making any change. This guarantees that the balance can never go negative through normal program flow and that every change to the balance is accompanied by a corresponding transaction history entry and a counter update. Encapsulation makes the User object a self-managing and self-protecting unit of data.

### **4.2 Abstraction:**

Abstraction means hiding the internal complexity of an operation and showing only a simple and clean interface to whoever is using it. The main() function in this project never needs to know how a deposit is processed internally. It simply calls u.deposit(amt) and waits for a true or false result. Behind that one function call, the deposit() function is doing several things at once: it checks whether the amount is valid, it adds the amount to the balance variable, it adds the same amount to the static totalSystemBalance variable, it calls addTransaction() to update the global transaction count, and it calls addHistory() to log a text entry into the private history array. All of this internal work is completely hidden from main(). The same idea applies to every other function in the User class. When main() calls u.showHistory(), it does not need to know that the function internally clears the screen, prints a formatted border, loops through the history array, prints each entry with a number, and handles the empty case with its own message. All of that complexity is hidden inside the function and the caller sees only a clean one-line call. Abstraction makes the main program much simpler and easier to read because all the detailed work is tucked away inside the class where it belongs, and the rest of the code only works with the high-level results.

### **4.3 Arrays of Objects:**

An array of objects means declaring multiple objects of a class together in a single array so that they can all be stored, accessed, and managed through index positions. In this project, the statement User users[100] in main() creates an array of 100 User objects. This array is the central and most important data structure in the entire program. Every registered wallet account exists as one element of this array. The five pre-registered accounts are placed at positions 0 through 4 when the program starts, and every new account created through the Create Wallet option is placed at the next available position tracked by the userCount variable. This array is used in almost every part of the program. The login module loops through the array and calls getName()

and `getPin()` on each element to find a match. The `searchUsers()` function takes the entire array as a parameter and loops through all elements checking name, account ID, and email against a search term. The Send Money feature accesses both the sender and the recipient by their index positions in the array and passes the recipient by reference to the `sendMoney()` function. The five initial users are also created as objects of the `User` class and assigned into the array using the parameterized constructor. Using an array of objects allows all accounts to be managed in one place with a simple loop, which is much cleaner than having a separate variable for each user.

## 4.4 Static Variables:

Static member variables are variables that belong to the class as a whole rather than to any individual object of the class. No matter how many `User` objects are created, there is only one copy of each static variable in memory and it is shared among all of them. In this project, three static member variables are declared inside the `User` class. The first one is `totalUsers` which is an integer that counts how many user accounts have been registered in the system. The second one is `totalTransactions` which is an integer that counts how many financial operations such as deposits, withdrawals, and money transfers have been completed across the entire system. The third one is `totalSystemBalance` which is a double that stores the sum of all wallet balances at any given moment. These three variables are defined outside the class in the global scope as required by C++ for static member variables, with the lines `int User::totalUsers = 0`, `int User::totalTransactions = 0`, and `double User::totalSystemBalance = 0`. They are updated through static member functions. Every time a new account is registered, the static `addUser()` function is called with the starting balance, and it increments `totalUsers` by one and adds the starting balance to `totalSystemBalance`. Every time a deposit, withdrawal, or transfer is successfully completed, the static `addTransaction()` function is called and it increments `totalTransactions` by one. The `deposit()` function also adds the deposited amount to `totalSystemBalance`, and the `withdraw()` function subtracts the withdrawn amount from it. Money transfers do not change `totalSystemBalance` because the amount leaving one account enters another so the total stays the same. The three static getter functions `getTotalUsers()`, `getTotalTransactions()`, and `getTotalSystemBalance()` are used by the System Info option to read and display these values on the screen.

## 4.5 Function overloading:

Function overloading is the ability to define two or more functions with the same name but with different parameter lists, and let the compiler decide which version to call based on what arguments are provided at the time of the call. This is a compile-time feature of C++ and it allows one function name to handle multiple related situations without needing different names for each version. In this project, function overloading is used in two places inside the `User` class. The first is the `addHistory()` function which has two versions. The first version takes only one string argument which is the transaction description and stores it directly into the history array. The second version takes two string arguments which are the description and an optional note, joins them together with the separator pipe `Note:` and stores the combined string. The second place where overloading is used is the `sendMoney()` function which also has two versions. The first version takes a reference to the recipient `User` object and the transfer amount, and it processes the complete transfer without any note. The second version takes the same two arguments plus an extra string for the note, and it includes that note in the history entries of both the sender and the receiver by calling the two-argument version of `addHistory()` for each of them. In `main()`, the correct version of `sendMoney()` is chosen automatically based on whether the user left the note field empty or typed something into it. If the note string is empty the first version is called, and if the note has content the second version is called.

This makes the code clean and natural because the same function name is used for both situations and the behavior adjusts based on the input.

## 4.6 Constructors:

A constructor is a special function inside a class that is called automatically when a new object of that class is created. Its job is to set the initial values of the object so that it starts in a valid and usable state. In this project, the User class has two constructors. The first is the default constructor which takes no arguments. It sets the name, email, and account ID to empty strings, the PIN to 0000, the balance to zero, and the historyCount to zero. This constructor is called automatically by C++ when the array User users[100] is declared in main(), because creating an array of objects requires the default constructor to initialize each of the 100 elements before the program uses them. The second is the parameterized constructor which takes five arguments: the name as a string, the PIN as a string, the email as a string, the account ID as a string, and an optional starting balance as a double that defaults to zero if not provided. This constructor is used in two places. The first is at the beginning of main() where the five pre-registered users are created with their specific names, PINs, emails, IDs, and starting balances. The second is inside the Create Wallet case where a new User object is built from the data the registering user just entered. Having both constructors means the class can handle both the situation where an empty placeholder object is needed and the situation where a fully initialized object is needed right away.

## 4.7 Getters and Setters:

Getters and setters are public member functions that provide controlled access to the private data members of a class. A getter function reads the value of a private variable and returns it to the caller without allowing the caller to change it. A setter function accepts a new value and assigns it to a private variable, and it can include validation logic before making the assignment. In this project, the User class has five getter functions. The getName() function returns the private name string. The getPin() function returns the private PIN string. The getEmail() function returns the private email string. The getAccountID() function returns the private accountID string. The getBalance() function returns the private balance double. All five of these are used throughout the program in different places. The login module in main() uses getName() and getPin() to compare values during authentication without touching the private variables directly. The searchUsers() function uses getName(), getAccountID(), and getEmail() to search through all users. The user menu display uses getName(), getAccountID(), and getBalance() to show the current user information at the top of the screen. The User class also has one setter which is setPin() that takes a new PIN string and assigns it directly to the private pin variable. Validation of the new PIN is done in main() using the isValidPin() function before setPin() is called, so the setter itself is kept simple. Together, getters and setters give the rest of the program a safe and clear way to interact with private data without ever breaking the encapsulation of the class.



## **5. Class Design:**

The entire system is built around a single class called User. This class contains all the data and all the behavior that a wallet account needs. The private section holds the following data members: name is a string for the account holder's full name, pin is a string for the four-digit PIN, email is a string for the email address, accountID is a string for the unique account identifier, balance is a double for the current wallet balance, history is a string array of size 200 for storing transaction entries, and historyCount is an integer tracking how many entries have been added to the history array. The three static variables totalUsers, totalTransactions, and totalSystemBalance are also declared in the private section and are shared across all User objects.

The public section contains the two constructors, four static functions for updating and reading the shared system data, five getter functions for safely reading private fields from outside the class, one setter function for updating the PIN, two overloaded versions of addHistory(), two overloaded versions of sendMoney(), the deposit() function, the withdraw() function, and the showHistory() function. Every function has one clear responsibility and together they cover everything any part of the program needs to do with a user account. The code in main() never directly accesses any private member and always goes through these public functions.

Key functions and what they do:

- deposit(double amt): Validates that the amount is greater than zero, adds it to the balance and to totalSystemBalance, calls addTransaction(), and logs a deposit entry in the history. Returns true on success and false if the amount is invalid.
- withdraw(double amt): Validates that the amount is greater than zero and not more than the balance, subtracts it from the balance and from totalSystemBalance, calls addTransaction(), and logs a withdrawal entry. Returns true on success and false on failure.
- sendMoney(User &recv, double amt): Validates that the amount is positive, not more than 50000, and not more than the sender's balance. Moves the money, calls addTransaction(), and logs entries for both the sender and the receiver without a note.
- sendMoney(User &recv, double amt, string note): Does everything the first version does but also includes the note in the history entries of both the sender and the receiver.
- showHistory(): Clears the screen, prints a formatted header with the user's name and account ID, and lists all entries in the history array with numbers. Shows a message if no transactions have been made yet.
- setPin(string p): Assigns the new PIN string directly to the private pin variable. Validation is handled in main() before this is called.

## **6. Utility Functions:**

Outside the User class, the program contains five global helper functions and one global search function. These are written as standalone functions outside any class and are called from main() wherever they are needed.

- toStr(double x): Converts a double value to a string with exactly two decimal places using sprintf() from the cstdio library. This is used every time an amount needs to be displayed on

screen or included as text in a history entry, since C++ does not automatically format doubles into readable strings.

- `lowerStr(string s)`: Converts every character in a string to lowercase using the `tolower()` function in a loop. This is used in the search feature so that searches work the same way regardless of whether the user types in uppercase or lowercase letters.
- `generateID(int index)`: Takes an integer index and produces a unique account ID string in the format AP followed by four digits, starting from AP1001 for the first user. It uses `sprintf()` to format the number with leading zeros. Each new user gets the ID that matches their position in the array.
- `isValidPin(const string &p)`: Returns true only if the input string is exactly four characters long and every character is a digit as checked by `isdigit()`. Used during registration and PIN change to reject any PIN that does not meet these conditions.
- `isEmailLower(const string &em)`: Returns false as soon as it finds any uppercase letter in the email string using `isupper()`. Returns true if the entire string passes. Used during registration to enforce the rule that email addresses must be all lowercase.

`searchUsers(User users[], int count, string term, int excludeIdx)`: Takes the full user array, the count of registered users, a search term, and the index of the currently logged-in user to be excluded. Converts the term to lowercase and checks every user's name, account ID, and email. Prints a numbered list of matching users, reads a selection safely using a stringstream, and returns the array index of the chosen user or negative one if the search was cancelled or no match was found.

## **7. Functional Modules:**

### **7.1 Registration and Login:**

The registration module starts when option 2 is selected from the main menu. The program asks the user to enter their name using `getline()` so that full names with spaces are captured correctly. The name field keeps prompting until a non-empty value is entered. The email is then read using `getline()` and passed to `isEmailLower()` which rejects it and asks again if any uppercase letter is found. The PIN is read using `cin` and checked by `isValidPin()` in a loop until a valid four-digit PIN is entered. At every field the user can type 0 to cancel and return to the main menu immediately. Once all three fields are accepted, a new User object is created using the parameterized constructor with the next auto-generated account ID, placed in the users array, and the `addUser()` static function is called to update the system totals. The login module reads the full name with `getline()` and the PIN with `cin`, then loops through the users array calling `getName()` and `getPin()` on each element through their getter functions to find a match. If a match is found the user menu is shown, otherwise an error message is displayed.

### **7.2 Wallet Operations:**

The Top-Up option reads the deposit amount as a string and uses a stringstream to safely convert it to a double. If the conversion fails the program shows an invalid amount message. If the amount is zero it is treated as a cancellation. If the amount passes these initial checks, the `deposit()` function of the logged-in User object is called. The `deposit()` function does the actual validation of whether the amount is positive, updates the balance, updates `totalSystemBalance`, counts the transaction, and logs the history entry.



The new balance is then shown using `getBalance()` through the getter function. The Cash-Out option works in the same way using the `withdraw()` function instead, which additionally checks that the amount does not exceed the current balance before making any change.

### **7.3 Money Transfer:**

The Send Money module begins by asking the logged-in user to type a search term. The `searchUsers()` function is called with this term and the full users array. It converts the term to lowercase and checks every user's name, account ID, and email using the getter functions `getName()`, `getAccountID()`, and `getEmail()`. A numbered list of all matching users is displayed and the sender picks one by number. After the recipient is selected, the sender enters a transfer amount which is validated using the same stringstream method. The sender is then given the option to type a note or press Enter to skip it. A confirmation screen shows all the details and the sender must type y to confirm. If the note field is empty the first version of `sendMoney()` is called. If the note has any content the overloaded version with the note parameter is called. The chosen function checks all conditions, moves the balance between the two User objects, and records the transaction in the history of both the sender and the receiver.

### **7.4 Transaction History:**

Every successful deposit, withdrawal, and money transfer adds a text string to the private history array inside the relevant User object by calling `addHistory()`. For money transfers, both the sender and the receiver get their own separate entry added to their own history array. When the user selects Transaction History from the menu, the `showHistory()` function of the logged-in User object is called. It clears the screen, prints a formatted header with the user's name and account ID, and then loops through the history array from index zero to `historyCount` minus one, printing each entry with a number. If `historyCount` is zero a message is printed saying there are no transactions yet. The history array holds a maximum of 200 entries per user and the `addHistory()` function checks this limit before adding each new entry.

### **7.5 PIN Management:**

The Change PIN option clears the screen and asks the currently logged-in user to enter a new four-digit PIN. The input is read with `cin` and passed to `isValidPin()`. If validation fails the user is asked to enter the PIN again in a loop. Typing 0 at any point cancels the operation and returns to the user menu. Once a valid PIN that is not 0 is entered, the `setPin()` setter function of the logged-in User object is called to update the private pin variable. A success message is shown before returning to the user menu. The next time this user logs in they will need to use the new PIN.

### **7.6 System Information:**

The System Info option is available from the main menu without requiring a login. It calls the three static getter functions of the User class which are `getTotalUsers()`, `getTotalTransactions()`, and `getTotalSystemBalance()`, and displays each value in a formatted output on the screen. The total users value shows how many accounts have been registered including the five pre-registered ones. The total transactions value shows how many deposits, withdrawals, and transfers have been completed since the program started. The total system balance shows the sum of all wallet balances at that moment. All three values are always accurate because the static variables are updated automatically every time a user is added or a transaction is processed anywhere in the program.

## 8. Input Validation:

Input validation is applied carefully at almost every point in the program to prevent crashes and make sure only correct data enters the system. For menu selections in both the main menu and the user menu, the input is read as a string first and then a stringstream is used to try converting it to an integer. If the conversion fails because the input contains non-numeric characters, or if there are leftover characters after the number which would prevent the stringstream from reaching the end of the string, the selection is treated as invalid and an error message is shown before displaying the menu again. This technique completely prevents the common C++ problem where a bad input corrupts the state of cin and causes the rest of the program to behave unpredictably.

For amount fields in Top-Up, Cash-Out, and Send Money, the same stringstream technique is applied to parse the entered text as a double. If parsing fails the program shows an invalid amount message and returns to the previous menu. If parsing succeeds but the result is zero the operation is treated as a cancellation. Negative values are caught by the deposit() and withdraw() functions which return false immediately if the amount is not greater than zero. The withdraw() function also rejects any amount greater than the current balance. The sendMoney() function adds a further check that the amount cannot exceed 50000.

For PIN fields during registration and the Change PIN operation, the isValidPin() function is called after every input attempt and a loop keeps asking until a valid four-digit PIN is entered or the user types 0 to cancel. For the email during registration, the isEmailLower() function scans every character and rejects the input if any uppercase letter is found, then immediately asks for the email again. For the name during registration, the program checks after every getline() call that the returned string is not empty and keeps prompting until a non-empty name is provided. At every field during registration the user can escape the process by typing 0 and the program returns to the main menu without creating any account.

## 9. Source Code:

```
#include <iostream> // For input/output operations like cin, cout
#include <string>    // For using string data type
#include <cstdio>    // For sprintf function
#include <cctype>    // For character functions like isdigit(), tolower()
#include <cstdlib>   // For system("cls") and system("pause")
#include <sstream>   // For stringstream to safely parse strings to numbers
using namespace std;
```

```
//Basic Utilite Functions
```

```
//Converts Double To String to easily add amount to history
```

```
//Because history is an string
```

```
string toStr(double x)
```

```
{
    char buf[50];
    sprintf(buf, "%.2f", x);
    return string(buf);
}
```

```
//Converts String to lower case for case-insensitive
string lowerStr(string s)
{
    for (int i = 0; i < (int)s.length(); i++)
        s[i] = tolower(s[i]);
    return s;
}

// Generates a unique account ID like AP1001, AP1002...
string generateID(int index)
{
    char buf[20];
    sprintf(buf, "AP%04d", index + 1001);
    return string(buf);
}

// Checks PIN to be exactly 4 digits
bool isValidPin(const string &p)
{
    if (p.length() != 4) return false;
    for (int i = 0; i < (int)p.length(); i++)
        if (!isdigit(p[i])) return false;
    return true;
}

// Checks Email to be only Lower case
bool isEmailLower(const string &em)
{
    for (int i = 0; i < (int)em.length(); i++)
        if (isupper(em[i])) return false;
    return true;
}

// USER CLASS
class User
{
    // Encapsulation: sensitive user data like pin account ID is kept private
    private:

    string name, pin, email, accountID;
    double balance;
    string history[200];
    int historyCount;

    // Static Variable (across the system)
    static int totalUsers;
    static int totalTransactions;
    static double totalSystemBalance;
```

public:

//Constructors

User()

```
{
    name = ""; pin = "0000"; email = "";
    accountID = ""; balance = 0; historyCount = 0;
}
```

User(string n, string p, string em, string id, double bal = 0)

```
{
    name = n; pin = p; email = em;
    accountID = id; balance = bal;
    historyCount = 0;
}
```

// Static Control

static void addUser(double bal)

```
{
    totalUsers++;
    totalSystemBalance += bal;
}
```

static void addTransaction() { totalTransactions++; }

static int getTotalUsers() { return totalUsers; }

static int getTotalTransactions() { return totalTransactions; }

static double getTotalSystemBalance() { return totalSystemBalance; }

// Getters & Setter: gets and sets values of the member through functions

string getName() { return name; }

string getPin() { return pin; }

string getEmail() { return email; }

string getAccountID() { return accountID; }

double getBalance() { return balance; }

void setPin(string p) { pin = p; }

// History Management

// Function overloading: two methods to add history entries with or without note

//without a note only balance

void addHistory(string entry)

```
{
    if (historyCount < 200)
        history[historyCount++] = entry;
}
```

```
//with a note
void addHistory(string entry, string note)
{
    if (historyCount < 200)
        history[historyCount++] = entry + " | Note: " + note;
}

// Wallet Operations
// Deposit money
bool deposit(double amt)
{
    if (amt <= 0) return false;
    balance += amt;
    totalSystemBalance += amt;
    addTransaction();
    addHistory("Deposited Rs." + toStr(amt));
    return true;
}

// Withdraw money
bool withdraw(double amt)
{
    if (amt <= 0 || amt > balance) return false;
    balance -= amt;
    totalSystemBalance -= amt;
    addTransaction();
    addHistory("Withdrew Rs." + toStr(amt));
    return true;
}

// Withdraw money from wallet
//without a note
bool sendMoney(User &recv, double amt)
{
    if (amt <= 0 || amt > 50000 || amt > balance) return false;

    balance -= amt;
    recv.balance += amt;
    addTransaction();

    addHistory("Sent Rs." + toStr(amt) +
        " to " + recv.name + " (" + recv.accountID + ")");

    recv.addHistory("Received Rs." + toStr(amt) +
        " from " + name + " (" + accountID + ")");

    return true;
}
```

//Overloaded with a note

```
bool sendMoney(User &recv, double amt, string note)
{
    if (amt <= 0 || amt > 50000 || amt > balance) return false;

    balance -= amt;
    recv.balance += amt;
    addTransaction();

    addHistory("Sent Rs." + toStr(amt) +
        " to " + recv.name + " (" + recv.accountID + ")", note);

    recv.addHistory("Received Rs." + toStr(amt) +
        " from " + name + " (" + accountID + ")", note);

    return true;
}
```

//Display History

```
void showHistory()
{
    system("cls");
    cout << "=====\n";
    cout << "    AsaanPay - Transaction History    \n";
    cout << "=====\n";
    cout << " " << name << " | " << accountID << "\n";
    cout << "=====\n";

    if (historyCount == 0)
        cout << " No transactions yet.\n";
    else
        for (int i = 0; i < historyCount; i++)
            cout << " " << i + 1 << ". " << history[i] << "\n";

    cout << "=====\n";
    system("pause");
}
};
```

// Static variables Declaration (Done Outside The class always)

```
int User::totalUsers = 0;
int User::totalTransactions = 0;
double User::totalSystemBalance = 0;
```



```
// Search Users Function
int searchUsers(User users[], int count, string term, int excludeIdx)
{
    string s = lowerStr(term);
    int idx[100], found = 0;

    for (int i = 0; i < count; i++)
    {
        if (i == excludeIdx) continue;

        string n = lowerStr(users[i].getName());
        string id = lowerStr(users[i].getAccountID());
        string em = lowerStr(users[i].getEmail());

        if (n.find(s) != string::npos ||
            id.find(s) != string::npos ||
            em.find(s) != string::npos)
            idx[found++] = i;
    }

    if (found == 0)
    {
        cout << "=====\n";
        cout << " No matching users found.\n";
        cout << "=====\n";
        system("pause");
        return -1;
    }

    cout << "=====\n";
    cout << "      SEARCH RESULTS\n";
    cout << "=====\n";
    for (int i = 0; i < found; i++)
    {
        cout << " " << i + 1 << ". " << users[idx[i]].getName() << "\n";
        cout << "   ID   : " << users[idx[i]].getAccountID() << "\n";
        cout << "   Email : " << users[idx[i]].getEmail() << "\n";
    }
    cout << " 0. Cancel\n";
    cout << "=====\n";
    cout << " Select: ";

    string input;
    int choice;
    cin >> input;
    stringstream ss(input);
    if (!(ss >> choice) || !(ss.eof()) || choice < 0 || choice > found)
        return -1;
}
```

```
    if (choice == 0) return -1;
    return idx[choice - 1];
}

// Main Program
int main()
{
    User users[100];
    int userCount = 0;
    int current = -1;

    //Initial Users
    users[0] = User("Sir Hassan Mujtaba", "1234", "hassanmujtaba@gmail.com", generateID(0), 20000);
    User::addUser(20000);

    users[1] = User("Badar Dhindsa", "8238", "badardhindsa@gmail.com", generateID(1), 5000);
    User::addUser(5000);

    users[2] = User("Fahad Malik", "8240", "fahadmalik@gmail.com", generateID(2), 5000);
    User::addUser(5000);

    users[3] = User("Talha Maqsood", "8385", "talhamaqsood@gmail.com", generateID(3), 5000);
    User::addUser(5000);

    users[4] = User("Hanzla", "6767", "hanzla@gmail.com", generateID(4), 5000);
    User::addUser(5000);

    userCount = 5;

    //loop: handles both main menu & user menu
    while (true)
    {
        system("cls");

        if (current == -1)
        {
            // MAIN MENU
            cout << "=====\n";
            cout << "    AsaanPay Digital Wallet\n";
            cout << "=====\n";
            cout << " 1. Login\n";
            cout << " 2. Create Wallet\n";
            cout << " 3. System Info\n";
            cout << " 0. Exit\n";
            cout << "=====\n";
            cout << " Choice: ";
        }
    }
}
```

```

string input;
int opt;
cin >> input;
stringstream ss(input);
if (!(ss >> opt) || !(ss.eof()))
{
    cout << "Invalid option.\n";
    system("pause");
    continue;
}

switch (opt)
{
case 0:
    system("cls");
    cout << "=====\n";
    cout << "  Thank you for using AsaanPay!\n";
    cout << "=====\n";
    return 0;

case 1:
    {
        system("cls");
        cout << "=====\n";
        cout << "      LOGIN\n";
        cout << "=====\n";
        cout << " (Enter 0 as name to cancel)\n";
        cout << "=====\n";

        string n, p;
        cin.ignore();
        cout << " Name : ";
        getline(cin, n);
        if (n == "0") break;

        cout << " PIN : ";
        cin >> p;
        if (p == "0") break;

        int found = -1;
        for (int i = 0; i < userCount; i++)
            if (users[i].getName() == n && users[i].getPin() == p)
                found = i;

        cout << "=====\n";
        if (found == -1)
            cout << " Invalid name or PIN.\n";
        else
            {

```

```

    current = found;
    cout << " Login Successful!\n";
    cout << " Welcome, " << users[current].getName() << "!\n";
}
cout << "===== \n";
system("pause");
break;
}

```

case 2:

```

{
system("cls");
cout << "===== \n";
cout << "      CREATE NEW WALLET\n";
cout << "===== \n";
cout << " (Enter 0 at any field to cancel)\n";
cout << "===== \n";

```

string n, em, p;

```

cin.ignore();
cout << " Name : ";
getline(cin, n);
if (n == "0") break;
while (n.empty())
{
    cout << " Name cannot be blank (0 to cancel): ";
    getline(cin, n);
    if (n == "0") break;
}
if (n == "0") break;

```

```

cout << " Email : ";
getline(cin, em);
if (em == "0") break;

```

```

while (!isEmailLower(em))
{
    cout << " Email cannot contain uppercase letters!\n";
    cout << " Enter again (0 to cancel): ";
    getline(cin, em);
    if (em == "0") break;
}
if (em == "0") break;

```

```

cout << " PIN  : ";
cin >> p;
if (p == "0") break;
while (!isValidPin(p))
{

```

```

    cout << " Must be exactly 4 digits (0 to cancel): ";
    cin >> p;
    if (p == "0") break;
}
if (p == "0") break;

string id = generateID(userCount);
users[userCount++] = User(n, p, em, id);
User::addUser(0);

cout << "=====\\n";
cout << " Wallet Created Successfully!\\n";
cout << " Name      : " << n << "\\n";
cout << " Account ID : " << id << "\\n";
cout << " Email      : " << em << "\\n";
cout << "=====\\n";
system("pause");
break;
}

case 3:
{
    system("cls");
    cout << "=====\\n";
    cout << "      SYSTEM INFORMATION\\n";
    cout << "=====\\n";
    cout << " Total Users      : " << User::getTotalUsers() << "\\n";
    cout << " Total Transactions : " << User::getTotalTransactions() << "\\n";
    cout << " Total Balance    : Rs. " << toStr(User::getTotalSystemBalance()) << "\\n";
    cout << "=====\\n";
    system("pause");
    break;
}

default:
    cout << "Invalid option.\\n";
    system("pause");
}
}

else
{
    // USER MENU
    User &u = users[current];

    cout << "=====\\n";
    cout << "  AsaanPay Digital Wallet\\n";
    cout << "=====\\n";
    cout << " User   : " << u.getName() << "\\n";
    cout << " ID     : " << u.getAccountID() << "\\n";

```

```

cout << " Balance : Rs. " << toStr(u.getBalance()) << "\n";
cout << "===== \n";
cout << " 1. Top-Up\n";
cout << " 2. Cash-Out\n";
cout << " 3. Send Money\n";
cout << " 4. Transaction History\n";
cout << " 5. Change PIN\n";
cout << " 0. Logout\n";
cout << "===== \n";
cout << " Choice: ";

string input;
int opt;
cin >> input;
stringstream ss(input);
if (!(ss >> opt) || !(ss.eof()))
{
    cout << "Invalid option.\n";
    system("pause");
    continue;
}

switch (opt)
{
case 0:
    system("cls");
    cout << "===== \n";
    cout << " Goodbye, " << u.getName() << "!\n";
    cout << " You have been logged out.\n";
    cout << "===== \n";
    system("pause");
    current = -1;
    break;

case 1:
    {
        // Top-Up
        system("cls");
        cout << "===== \n";
        cout << "      TOP-UP\n";
        cout << "===== \n";
        cout << " Amount (0 to cancel): ";
        string amtStr;
        double amt;
        cin >> amtStr;
        stringstream ssAmt(amtStr);
        if (!(ssAmt >> amt) || !(ssAmt.eof())) { cout << "Invalid amount.\n"; system("pause"); break; }

        cout << "===== \n";
    }
}

```

```

if (amt == 0)
    cout << " Cancelled.\n";
else if (!u.deposit(amt))
    cout << " Invalid amount.\n";
else
    {
        cout << " Top-Up Successful!\n";
        cout << " New Balance: Rs. " << toStr(u.getBalance()) << "\n";
    }
cout << "=====\n";
system("pause");
break;
}

case 2:
    {
        // Cash-Out
        system("cls");
        cout << "=====\n";
        cout << "          CASH-OUT\n";
        cout << "=====\n";
        cout << " Amount (0 to cancel): ";
        string amtStr;
        double amt;
        cin >> amtStr;
        stringstream ssAmt(amtStr);
        if (!(ssAmt >> amt) || !(ssAmt.eof())) { cout << "Invalid amount.\n"; system("pause"); break; }

        cout << "=====\n";
        if (amt == 0)
            cout << " Cancelled.\n";
        else if (!u.withdraw(amt))
            cout << " Insufficient balance or invalid amount.\n";
        else
            {
                cout << " Cash-Out Successful!\n";
                cout << " New Balance: Rs. " << toStr(u.getBalance()) << "\n";
            }
        cout << "=====\n";
        system("pause");
        break;
    }

case 3:
    {
        // Send Money
        system("cls");
        cout << "=====\n";
        cout << "          SEND MONEY\n";
        cout << "=====\n";
    }

```



```

cout << " Search: ";

cin.ignore();
string s;
getline(cin, s);

if (s == "0") break;

int idx = searchUsers(users, userCount, s, current);
if (idx == -1) break;

User &recv = users[idx];

system("cls");
cout << "=====\\n";
cout << "          SEND MONEY\\n";
cout << "=====\\n";
cout << " To    : " << recv.getName() << "\\n";
cout << " ID    : " << recv.getAccountID() << "\\n";
cout << "=====\\n";
cout << " Amount (0 to cancel): ";

string amtStr;
double amt;
cin >> amtStr;
stringstream ssAmt(amtStr);
if (!(ssAmt >> amt) || !(ssAmt.eof())) { cout << "Invalid amount.\\n"; system("pause"); break; }

if (amt == 0) { cout << "Cancelled.\\n"; system("pause"); break; }

cout << " Note (Enter to skip): ";
cin.ignore();
string note;
getline(cin, note);

cout << "=====\\n";
cout << " To    : " << recv.getName() << " (" << recv.getAccountID() << ")\\n";
cout << " Amount : Rs. " << toStr(amt) << "\\n";
if (note != "")
    cout << " Note : " << note << "\\n";

cout << "=====\\n";
cout << " Confirm? (y/n): ";
char c; cin >> c;

if (c != 'y' && c != 'Y')
    cout << " Transaction Cancelled.\\n";
else
    {
        bool ok;
    }

```

```

    if (note == "")
        ok = u.sendMoney(recv, amt);
    else
        ok = u.sendMoney(recv, amt, note);

    if (!ok)
        cout << " Failed. Check amount or balance.\n";
    else
        cout << " Money Sent Successfully!\n"
            << " New Balance: Rs. " << toStr(u.getBalance()) << "\n";
    }
    cout << "===== \n";
    system("pause");
    break;
}

case 4: u.showHistory(); break;
case 5:
{
    // Change PIN
    system("cls");
    cout << "===== \n";
    cout << "          CHANGE PIN\n";
    cout << "===== \n";

    string oldPin;
    cout << " Enter OLD PIN (0 to cancel): ";
    cin >> oldPin;

    if (oldPin == "0")
    {
        cout << " Cancelled.\n";
        system("pause");
        break;
    }

    // Verify old PIN
    if (oldPin != u.getPin())
    {
        cout << "===== \n";
        cout << " Incorrect old PIN!\n";
        cout << "===== \n";
        system("pause");
        break;
    }

    // Ask for new PIN
    string newPin, confirmPin;

    cout << " Enter NEW 4-digit PIN (0 to cancel): ";

```

```
cin >> newPin;

if (newPin == "0")
{
    cout << " Cancelled.\n";
    system("pause");
    break;
}

while (!isValidPin(newPin))
{
    cout << " Invalid PIN. Must be 4 digits (0 to cancel): ";
    cin >> newPin;

    if (newPin == "0")
    {
        cout << " Cancelled.\n";
        system("pause");
        break;
    }
}
if (newPin == "0") break;

// Confirm new PIN
cout << " Confirm NEW PIN: ";
cin >> confirmPin;

if (confirmPin != newPin)
{
    cout << "=====\n";
    cout << " PINs do NOT match! PIN not changed.\n";
    cout << "=====\n";
    system("pause");
    break;
}
// Change PIN
u.setPin(newPin);
cout << " PIN Changed Successfully!\n";
cout << "=====\n";
system("pause");
break;
}
default:
    cout << "Invalid option.\n";
    system("pause");
}
}
}
return 0;
}
```

## 10. Sample Output:

### Main Screen:

```
=====
      AsaanPay Digital Wallet
=====
1. Login
2. Create Wallet
3. System Info
0. Exit
=====
Choice:
```

### Create Wallet Screen:

```
=====
      CREATE NEW WALLET
=====
(Enter 0 at any field to cancel)
=====
Name   : Abdullah
Email  : Abdullah@gmail.com
Email cannot contain uppercase letters!
Enter again (0 to cancel): abdullah@gmail.com
PIN    : 1661
=====
Wallet Created Successfully!
Name       : Abdullah
Account ID : AP1006
Email      : abdullah@gmail.com
=====
Press any key to continue . . . |
```

### Login Screen:

```
=====
                        LOGIN
=====
(Enter 0 as name to cancel)
=====
Name : Abdullah
PIN  : 1661
=====
Login Successful!
Welcome, Abdullah!
=====
Press any key to continue . . .
```

### Logged in Screen:

```
=====
          AsaanPay Digital Wallet
=====
User      : Abdullah
ID        : AP1006
Balance   : Rs. 0.00
=====
1. Top-Up
2. Cash-Out
3. Send Money
4. Transaction History
5. Change PIN
0. Logout
=====
Choice:
```

### Top-up Option:

```
=====
                        TOP-UP
=====
Amount (0 to cancel): 10000
=====
Top-Up Successful!
New Balance: Rs. 10000.00
=====
Press any key to continue . . . |
```

### Cash-Out Option:

```
=====
                        CASH-OUT
=====
Amount (0 to cancel): 1000
=====
Cash-Out Successful!
New Balance: Rs. 9000.00
=====
Press any key to continue . . .
```

**Send Money Option:**

**Search User:**

```
=====
                        SEND MONEY
=====
Search: Badar
=====
                        SEARCH RESULTS
=====
1. Badar Dhindsa
   ID      : AP1002
   Email   : badardhindsa@gmail.com
0. Cancel
=====
Select: 1|
```

**Send without note:**

```
=====
                        SEND MONEY
=====
To      : Badar Dhindsa
ID      : AP1002
=====
Amount (0 to cancel): 5000
Note (Enter to skip):
=====
To      : Badar Dhindsa (AP1002)
Amount : Rs. 5000.00
=====
Confirm? (y/n): y
Money Sent Successfully!
New Balance: Rs. 4000.00
=====
Press any key to continue . . . |
```

### With Note:

```
=====
                        SEND MONEY
=====
To      : Badar Dhindsa
ID      : AP1002
=====
Amount (0 to cancel): 2000
Note (Enter to skip): Happy Birthday
=====
To      : Badar Dhindsa (AP1002)
Amount  : Rs. 2000.00
Note    : Happy Birthday
=====
Confirm? (y/n): y
Money Sent Successfully!
New Balance: Rs. 2000.00
=====
Press any key to continue . . . |
```

### Transaction History Option:

```
=====
      AsaanPay - Transaction History
=====
Abdullah | AP1006
=====
1. Deposited Rs.10000.00
2. Withdrew Rs.1000.00
3. Sent Rs.5000.00 to Badar Dhindsa (AP1002)
4. Sent Rs.2000.00 to Badar Dhindsa (AP1002) | Note: Happy Birthday
=====
Press any key to continue . . .
```



### Change Pin:

```
=====
                        CHANGE PIN
=====
Enter OLD PIN (0 to cancel): 1661
Enter NEW 4-digit PIN (0 to cancel): 5252
Confirm NEW PIN: 5252
PIN Changed Successfully!
=====
```

### Log-Out Option:

```
=====
Goodbye, Abdullah!
You have been logged out.
=====
Press any key to continue . . .
```

### System Information Option:

```
=====
                        SYSTEM INFORMATION
=====
Total Users           : 6
Total Transactions    : 4
Total Balance         : Rs. 49000.00
=====
Press any key to continue . . .
```

### Exit Option:

```
=====
Thank you for using AsaanPay!
=====
```

## **11. Limitations:**

The current version of the system has the following limitations:

- All user data and transaction records are stored only in memory inside arrays. Everything is permanently lost when the program closes because there is no file saving or database, and every run of the program starts fresh with only the five pre-registered accounts.
- The user array has a fixed compile-time size of 100 and the transaction history array inside each User object has a fixed size of 200. These limits cannot grow at runtime and if either is exceeded the program simply stops accepting new entries without clearly alerting the user.
- The PIN is stored as a plain string with no hashing or encryption. If someone had access to the program's memory they could read the PINs directly, which would be a serious security issue in any real application.
- Login is matched using the full name string and the PIN. There is no unique username field, so if two registered users happen to share the exact same full name and the same PIN, the first one found in the array would always be the one that gets logged in.
- The maximum transfer limit of 50000 is hardcoded inside the `sendMoney()` function and cannot be adjusted without editing and recompiling the source code.
- The program is a single-threaded console application that can only handle one active user session at a time.

## **12. Future Enhancements:**

The following improvements could be made to the system in the future:

- A file-based or database storage system could be added so that all user accounts and transaction records are saved permanently and remain available across multiple runs of the program.
- A graphical user interface could replace the console menus to make the application more accessible and comfortable for users who are not familiar with terminal programs.
- The PIN and other sensitive user data could be hashed using a standard algorithm before being stored so that raw values are never kept in memory or on disk in a readable form.
- A unique username field could be added to the User class so that login does not depend on matching a full name string, which would fix the issue that occurs when two users share the same name.
- The fixed-size arrays could be replaced with dynamic structures such as vectors so that the system can hold any number of users and any number of transaction entries without hitting a hard limit.
- An export feature could be added so that users can save their transaction history to a text file they can keep or share.

## 13. Conclusion:

The AsaanPay Digital Wallet System is a complete and fully working console application written in C++ that simulates the core features of a real digital wallet. Users can register accounts, log in with a PIN, top up and cash out their balance, send money to other users with an optional note, view their complete transaction history, and change their PIN. The main menu also provides a system information screen that uses static class data to display live statistics about the entire system.

The project demonstrates seven Object-Oriented Programming concepts through actual working code. Encapsulation keeps all private user data inside the User class and prevents direct access from outside. Abstraction hides the internal complexity of every operation behind clean and simple function calls so that main() stays readable and focused. Arrays of Objects provide the central data structure through which all 100 possible User accounts are stored, searched, and accessed by index. Static Variables maintain the three system-wide counters inside the class and keep them accurate as users are added and transactions take place. Function Overloading is used for both addHistory() and sendMoney() where two versions of each function exist and the correct one is chosen automatically based on whether a note is provided. Constructors properly initialize every User object whether it is being created as an empty placeholder in the array or as a fully registered account with real data. Getters and Setters provide safe and controlled access to private data fields so that the rest of the program can read account information and update the PIN without ever breaking the encapsulation of the class.

The project successfully achieves everything it was designed to do. The code is well organized, every part has a clear responsibility, and all seven OOP concepts are applied in ways that genuinely improve the structure and reliability of the program. Even though the system has limitations that come from its academic scope, it provides a solid and honest example of how Object-Oriented Programming makes a program easier to build, read, and maintain.